

Lab 01 - TCP/IP Behavior and Resource Exhaustion

Name: Syed Saad Imran

Student ID: 09370

1.1 Objectives

By the end of this lab, students will be able to:

- Observe TCP connection establishment using the three-way handshake.
- Capture and analyze TCP packets using Wireshark.
- Understand how TCP servers allocate resources per connection.
- Demonstrate TCP resource exhaustion in a controlled environment.

1.2 Introduction

TCP is a connection-oriented transport protocol that ensures reliable communication between hosts. Before any application data is exchanged, a logical connection must be established between the client and the server.

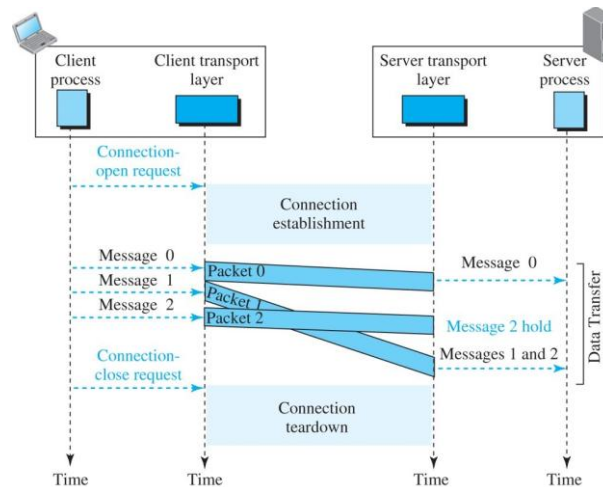


Figure 1: Connection-oriented service showing connection establishment, data transfer, and termination

Figure 1 highlights the connection-oriented nature of TCP communication. Before data transfer begins, a connection must be established between the communicating hosts. Once the connection is active, data can be exchanged reliably in both directions, and the connection is explicitly terminated when communication ends. This structured behavior distinguishes TCP from connectionless protocols such as UDP.

Before data transfer begins, TCP establishes a connection using the *three-way handshake*, which involves the exchange of SYN, SYN-ACK, and ACK packets.

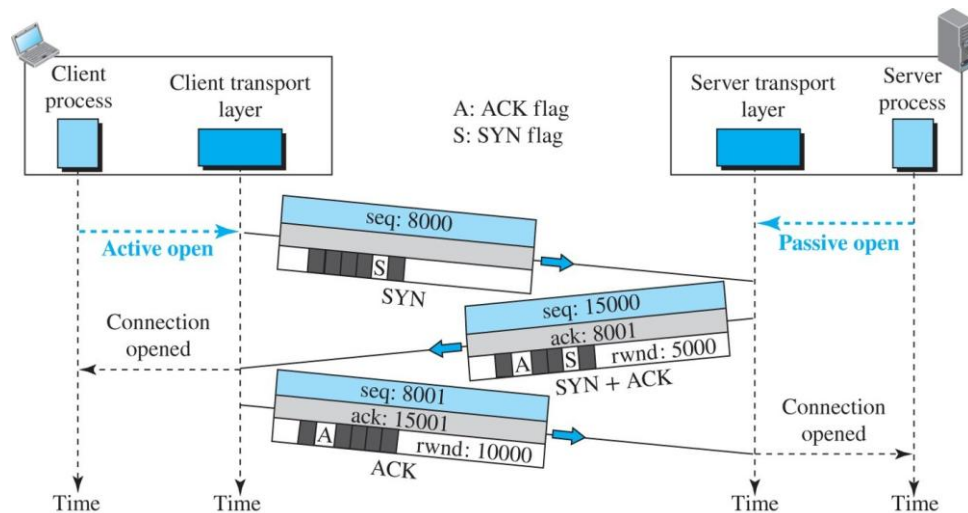


Figure 2: TCP three-way handshake illustrating SYN, SYN-ACK, and ACK packet exchange

Figure 2 illustrates the TCP three-way handshake process. The client initiates the connection by sending a SYN packet. The server responds with a SYN-ACK packet, acknowledging the request. Finally, the client replies with an ACK packet, confirming the connection. After this exchange, both endpoints are synchronized and ready for reliable data transfer.

Ethics Reminder: All experiments must be performed only on the provided lab virtual machines. Never run these experiments on public or production systems.

1.3 Lab Environment

Software Installation

Perform the following installations on the respective virtual machines.

On VM1 (Client / Analyzer):

```
sudo apt update
sudo apt install python3 wireshark tcpdump net-tools -y
```

On VM2 (Server):

```
sudo apt update
sudo apt install python3 net-tools -y
```

Virtual Machines

Two Linux virtual machines are used:

- **VM1 (Client / Analyzer)** – runs Wireshark and client scripts
- **VM2 (Server)** – runs TCP server programs

Both machines are connected to the same private virtual network.



Finding the Server IP Address

On **VM2**, determine the IP address by running:

```
hostname -I
```

Use this IP address as the SERVER IP in all client programs.

192.168.254.133

1.4 Task 1: TCP Three-Way Handshake

Task Overview

In this task, you will establish a TCP connection between a client and server and observe the three-way handshake using Wireshark.

Step 1: TCP Server (VM2)

Create and run the server program.

```
mkdir tcp_lab  
cd tcp_lab  
nano tcp_server_task1.py
```

```
import socket  
  
SERVER_IP = "0.0.0.0"  
SERVER_PORT = 5000  
  
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  
server.bind((SERVER_IP, SERVER_PORT))  
server.listen(1)  
print("Server listening on port", SERVER_PORT)  
conn, addr = server.accept()  
print("Connection received from", addr)  
data = conn.recv(1024)  
print("Received:", data.decode(errors="replace"))  
  
conn.close()  
server.close()
```

Listing 1: TCP Server – Task 1

Run the server:



```
python3 tcp_server_task1.py
```

Step 2: Start Wireshark (VM1)

- Launch Wireshark with administrator privileges
- Select the active network interface
- Apply the following display filter

```
tcp.port == 5000
```

No.	Time	Source	Destination	Protocol	Length	Info
45	46.214339445	192.168.254.132	192.168.254.134	TCP	74	53346 → 5000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK
46	46.214822066	192.168.254.134	192.168.254.132	TCP	74	5000 → 53346 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MS
47	46.214859298	192.168.254.132	192.168.254.134	TCP	66	53346 → 5000 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2
48	46.215562732	192.168.254.132	192.168.254.134	TCP	82	53346 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=16 T
49	46.215868627	192.168.254.132	192.168.254.134	TCP	66	53346 → 5000 [FIN, ACK] Seq=17 Ack=1 Win=64256 Len=0
50	46.216267784	192.168.254.134	192.168.254.132	TCP	66	5000 → 53346 [ACK] Seq=1 Ack=17 Win=65152 Len=0 TSval=
51	46.219334622	192.168.254.134	192.168.254.132	TCP	66	5000 → 53346 [FIN, ACK] Seq=1 Ack=18 Win=65152 Len=0
52	46.219354144	192.168.254.132	192.168.254.134	TCP	66	53346 → 5000 [ACK] Seq=18 Ack=2 Win=64256 Len=0 TSval=

Figure 3: Wireshark capture showing the TCP three-way handshake. The client initiates the connection with a SYN packet, the server responds with a SYN-ACK, and the client completes the handshake with an ACK. This exchange establishes a reliable TCP connection before data transfer begins.

Step 3: TCP Client (VM1)

```
nano tcp_client_task1.py
```

```
import socket

SERVER_IP = "REPLACE_WITH_VM2_IP"
SERVER_PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, SERVER_PORT))
client.sendall(b"Hello TCP Server")
client.close()
```

Listing 2: TCP Client – Task 1

Run the client:

```
python3 tcp_client_task1.py
```

Attach screenshot clearly showing the SYN, SYN-ACK, and ACK packets here

```
Server listening on port 5000
Connection received from ('192.168.254.132', 53458)
Received: Hello TCP Server
```

Capturing from ens33						
No.	Time	Source	Destination	Protocol	Length	Info
11	56.839266819	192.168.254.132	192.168.254.133	TCP	74	54852 → 5000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK
12	56.840488520	192.168.254.133	192.168.254.132	TCP	74	5000 → 54852 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MS
13	56.840545907	192.168.254.132	192.168.254.133	TCP	66	54852 → 5000 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2
14	56.840836052	192.168.254.132	192.168.254.133	TCP	82	54852 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=16 T
15	56.840931041	192.168.254.132	192.168.254.133	TCP	66	54852 → 5000 [FIN, ACK] Seq=17 Ack=1 Win=64256 Len=0 T
16	56.841565104	192.168.254.133	192.168.254.132	TCP	66	5000 → 54852 [ACK] Seq=1 Ack=17 Win=65152 Len=0 TSval=
17	56.841565340	192.168.254.133	192.168.254.132	TCP	66	5000 → 54852 [FIN, ACK] Seq=17 Ack=17 Win=65152 Len=0 T
18	56.841595759	192.168.254.132	192.168.254.133	TCP	66	54852 → 5000 [ACK] Seq=18 Ack=2 Win=64256 Len=0 TSval=
19	56.841865055	192.168.254.133	192.168.254.132	TCP	66	5000 → 54852 [ACK] Seq=2 Ack=18 Win=65152 Len=0 TSval=

Briefly explain the role of each step in the TCP three-way handshake below

Packet 1 (SYN): Client sends a packet with the SYN flag to start a connection to the server and synchronize sequence numbers.

Packet 2 (SYN + ACK): The server receives the request and then replies with the packet with both the SYN and ACK

Packet 3 (ACK): Client sends the final ACK packet to acknowledge receipt of the servers SYN. Connection is complete and data transfer can start.

1.5 Task 2: TCP Resource Exhaustion

Figure 4 shows the send and receive buffers maintained by TCP at both the client and server sides. Each active TCP connection requires dedicated buffer space and control structures to store outgoing and incoming data. As the number of simultaneous connections increases, the server must allocate more memory and processing resources to manage these buffers. In Task 2, students demonstrate how holding multiple connections open can exhaust available server resources, leading to delayed responses or rejected connection attempts.

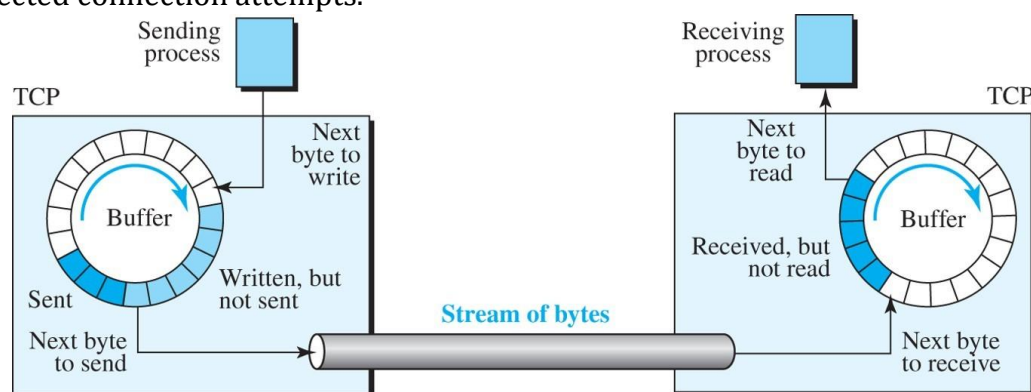


Figure 4: TCP send and receive buffers allocated per connection, contributing to resource usage on the server

Task Overview

This task demonstrates how a server with limited capacity behaves when subjected to many connection attempts.



Step 1: Limited-Backlog Server (VM2)

```
nano limited_server_task2.py
```

```
import socket
import time

SERVER_IP = "0.0.0.0"
SERVER_PORT = 5001

server=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET,socket.SO_REUSEADDR, 1)
server.bind((SERVER_IP,SERVER_PORT))
server.listen(2)
print("Limited backlog server listening on port", SERVER_PORT)

connections = []
end_time = time.time()+ 30

while time.time() < end_time:
    server.settimeout(1)
    try:
        conn, addr =server.accept()
        print("Accepted connection
        from",addr)
        connections.append(conn)
    except socket.timeout:
        pass

for c in connections:
    c.close()
server.close()
```

Listing 3: Limited-Backlog Server – Task 2

Run the server:

```
python3 limited_server_task2.py
```

Step 2: Multiple Connections Client (VM1)

```
nano multi_client_task2.py
```



```
import socket
import time

SERVER_IP = "REPLACE_WITH_VM2_IP"
SERVER_PORT = 5001
connections = []

for i in range(10):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((SERVER_IP, SERVER_PORT))
        connections.append(s)
        print("Opened connection", i + 1)
    except ConnectionRefusedError:
        print("Connection refused at attempt", i + 1)
        break

time.sleep(30)

for s in connections:
    s.close()
```

Listing 4: Multi-Client Script – Task 2

Run the client:

```
python3 multi_client_task2.py
```

Attach screenshot showing multiple client connection attempts here

```
administrator@administrator-virtual-machine:~$ python3 multi_client_task2.py
Opened connection 1
Opened connection 2
Opened connection 3
Opened connection 4
Opened connection 5
Opened connection 6
Opened connection 7
Opened connection 8
Opened connection 9
Opened connection 10
```

Attach screenshot of netstat -tn output here



Active Internet connections (w/o servers)					
Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	192.168.254.132:35636	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35586	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35596	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35574	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35650	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35646	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35642	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35624	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35604	192.168.254.133:5001	ESTABLISHED
tcp	0	0	192.168.254.132:35614	192.168.254.133:5001	ESTABLISHED



Task 2 Reflection: TCP Resource Exhaustion

Answer the following questions based on your observations from Task 2.

Q1. What happens to new client connection attempts once the server backlog limit is reached?

New attempts are rejected and the client receives a connection refused error as the server can't accept new requests

Q2. Why does the server start refusing or dropping connections after a certain point?

Server code sets a limit or backlog on how many connections can wait in line at once. When the waiting list is full, the server refuses new connections

Q3. How does holding established TCP connections consume server resources?

Every active connection requires the server to reserve a piece of memory to send and receive data. If there are too many connections then the server runs out of memory

Q4. Briefly explain how this behavior can be exploited in denial-of-service (DoS) attacks.

Attacker can overload the server with a lot of fake connection requests which fills up the servers memory until it run

Q5. Name one practical mitigation technique used in real systems to reduce TCP resource exhaustion.

SYN cookies where the server doesn't reserve memory for a connection until the final ACK message which prevents attacker from filling up the memory with fake requests.



Assessment Rubric Lab 01: TCP/IP Behavior and Resource Exhaustion

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 5 (Results)	LR 10 (Analysis)
In-Lab	Task 1 (TCP Handshake)	/20	/20
	Task 2 (Resource Exhaustion)	/20	/20
Total Points		/40	/40
CLO Mapped		CLO 1	

Affective Domain Rubric		Points	CLO Mapped CLO 1
AR7	Report Submission	/20	

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.

Lab Evaluation Assessment Rubric

ID	Assessment Element	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
LR5	Results & Plots	Figures/graphs/tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some details are missing.	Figures/graphs/tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR9	Report	All the in-lab tasks are not included in report and/or the report is submitted too late.	Most tasks are included in report but not well explained. Necessary figures/plots are not included. Report is submitted after due date.	Good summary of most inlab tasks included. Work supported by figures/plots with explanations. Report submitted timely.	Detailed summary of inlab tasks provided. All tasks included and explained well. Data clearly presented including necessary figures, plots and tables.
LR10	Analysis	Results are not interpreted or analyzed incorrectly.	Analysis of obtained results is incomplete. Conclusions are not drawn.	Results are analyzed and concluded but not well explained; justification is not provided with reasoning.	Results are well analyzed supported with reasons. Good comparison between theoretical and experimental results with correct and useful conclusion.

Affective Domain Rubric

ID	Assessment Element	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
----	--------------------	-------------------------	---------------------	---------------	--------------------



AR7	Report Content/- Code comments	Most of the questions are not answered / figures are not labelled/ titles are not mentioned / units are not mentioned. No comments are present in the code.	Some of the questions are answered, figures are labelled, titles are mentioned and units are mentioned. Few comments are stated in the code.	Majority of the questions are answered, figures are labelled, titles are mentioned and units are mentioned. Comments are stated in the code.	All the questions are answered, figures are labelled, titles are mentioned and units are properly mentioned. Proper comments are stated in the code.
-----	-----------------------------------	---	--	--	--